

UNIVERSIDAD POLITÉCNICA DE PACHUCA

INGENIERÍA EN SOFTWARE

Inteligencia Artificial

Proyecto Final: Optimización de Salidas de Combis Predicción de Demanda y Aprendizaje por Exploración/Explotación (UCB)

ALUMNOS:

Angeles Sanchez Ximena Nathaly
Islas Lozada Emmanuel

CATEDRÁTICA:

Dra. Anabel Martínez Vargas

IA UTILIZADA:

Antigravity (Gemini 3.6 Flash - Google DeepMind)

Pachuca de Soto, Hidalgo, 20 de agosto de 2026



Índice

1. Parte 1: Planteamiento del Problema y Objetivos

En el sistema de transporte colectivo universitario que conecta a la comunidad académica con el campus de la Universidad Politécnica de Pachuca (UPP), operan dos rutas principales con comportamientos operacionales contrastantes:

1. **Ruta Central de Autobuses / Las Vías:** Conecta nodos urbanos de alto tráfico. Registra una afluencia constante e intensa de estudiantes con intervalos de llegada menores a 1 minuto por alumno durante horarios pico.
2. **Ruta Puente de Tuzos (Mineral de la Reforma):** Presenta una menor densidad poblacional estudiantil y un flujo más lento, con una llegada promedio de **1 alumno cada 7 minutos** ($\lambda \approx 0,143$ alumnos/minuto) durante franjas horarias normales.

1.1. Problemática Operativa

Actualmente, las unidades operan bajo una regla empírica rígida: *no salir hasta llenar los 18 asientos de la combi*.

Durante las semanas de fin de cuatrimestre, este criterio provoca salidas muy tardías en la ruta Puente de Tuzos. Los estudiantes que abordan primero sufren tiempos de espera acumulados superiores a 60 u 80 minutos en la parada, causando retrasos críticos en sus clases y exámenes finales.

1.2. Objetivo General del Proyecto

Diseñar e implementar un sistema inteligente basado en Ciencia de Datos y Aprendizaje por Refuerzo que recomiende la ventana óptima de despacho para cada unidad, minimizando el tiempo de espera de los alumnos y garantizando un nivel razonable de ocupación operacional.

Componentes Fundamentales de la Solución

- **Generación de Dataset Simulado:** Modelación estocástica por procesos de Poisson ajustados a 21 días de demanda.
- **Regresión Lineal Simple:** Modelos Mínimos Cuadrados Ordinarios (OLS) para estimación de tiempos de llenado y recorrido.
- **Algoritmo UCB1 (Multi-Armed Bandit):** Aprendizaje por exploración/explotación para seleccionar políticas dinámicas de despacho.

2. Parte 2: Simulación y Generación del Dataset

Para simular de manera estocástica y reproducible las 3 semanas de fin de cuatrimestre (21 días de operación continua), se dividió la jornada laboral (06:00 a 20:00 horas) en intervalos discretos de 15 minutos.

2.1. Proceso Estocástico de Llegada de Alumnos (Poisson)

La llegada de alumnos por intervalo de 15 minutos (X_t) en cada ruta se modela como una variable aleatoria Poisson:

$$X_t \sim \text{Poisson}(\lambda_{\text{efectivo}}) \quad (1)$$

donde la tasa de llegada efectiva ($\lambda_{\text{efectivo}}$) se expresa según la tasa base (λ_{base}), la duración del intervalo ($\Delta t = 15$ min) y un multiplicador de hora pico (M_{pico}):

$$\lambda_{\text{efectivo}} = \lambda_{\text{base}} \times \Delta t \times M_{\text{pico}} \quad (2)$$

Parámetros de Entrada por Ruta

- **Puente de Tuzos:** $\lambda_{\text{base}} = \frac{1}{7}$ alumnos/min ($\approx 2,14$ pax/15 min). $M_{\text{pico}} = 1,6$. Tiempo medio de viaje: 20 min.
- **Central de Autobuses / Las Vías:** $\lambda_{\text{base}} = \frac{1}{3}$ alumnos/min ($\approx 5,0$ pax/15 min). $M_{\text{pico}} = 3,5$ (< 1 min por alumno). Tiempo medio de viaje: 27.5 min.

2.2. Estructura del Dataset Resultante

El dataset simulado se exporta como archivo CSV (`data/demand_dataset.csv`) con la siguiente especificación de columnas:

Tabla 1: Estructura del Dataset Simulado de Demanda y Despachos (13 Campos Rúbrica)

Categoría Rúbrica	Columna	Descripción Técnica
Demanda	fecha, hora	Estampa de tiempo del intervalo (String / Timestamp).
	ruta	Identificador de ruta (<i>Puente Tuzos</i> vs <i>Las Vías</i>).
	alumnos_esperando	Conteo acumulado de alumnos en la parada principal.
	llegadas_intervalo	Alumnos que arribaron en los 15 min actuales (Poisson).
Despacho	hora_salida	Estampa de tiempo efectiva de partida (o N/A si no salió).
	pasajeros_al_salir	Total de pasajeros a bordo ($0 \leq N \leq 18$).
	tipo_salida	Categorica: <i>Llena</i> (18 pax), <i>Parcial</i> (< 18) o <i>Ninguna</i> .
	tiempo_espera_acum	Tiempo de espera acumulado del primer pasajero (minutos).
Ruta	tiempo_recorrido	Duración del trayecto a la UPP en minutos.
	paradas_intermedias	Número de estaciones intermedias configuradas.
	alumnos_recogidos_intermedias	Pasajeros abordados durante las paradas del trayecto.
	hora_llegada_upp	Estampa de tiempo de arribo a la universidad (o N/A).

3. Parte 3: Modelo de Regresión Lineal Simple

3.1. Ecuación del Modelo Teórico y Estimadores OLS

El modelo probabilístico de regresión lineal simple responde a la formulación:

$$Y_i = a + b \cdot X_i + e_i, \quad e_i \sim \mathcal{N}(0, \sigma^2) \quad (3)$$

Los estimadores de Mínimos Cuadrados Ordinarios (OLS) para la intersección (a) y la pendiente (b) se calculan mediante:

Formulación de Estimadores OLS

$$b = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}, \quad a = \bar{Y} - b\bar{X} \quad (4)$$

Se evalúa específicamente el **Modelo 2 (Tiempo para Llenar Combi vs Alumnos Esperando)**:

$$\text{minutos_para_llenar} = a + b \times \text{alumnos_esperando} \quad (5)$$

3.2. Contraste de Hipótesis Estadísticas

Para evaluar la significancia del coeficiente de la pendiente (b), se plantea el siguiente contraste de hipótesis bilateral:

Tabla 2: Planteamiento de Hipótesis Estadísticas sobre la Pendiente

Hipótesis	Formulación	Interpretación Contextual
Hipótesis Nula (H_0)	$H_0 : b = 0$	El número de alumnos en espera no influye en el tiempo restante para llenar la unidad.
Hipótesis Alternativa (H_1)	$H_1 : b \neq 0$	El número de alumnos en espera reduce significativamente el tiempo necesario para llenar la unidad.

3.3. Verificación de Supuestos del Modelo OLS

Para validar la inferencia del modelo, se analizan los residuos $e_i = Y_i - \hat{Y}_i$:

1. **Normalidad de Residuos (Prueba de Shapiro-Wilk):** Evalúa si los errores provienen de una distribución normal. Se acepta el supuesto si $p_{\text{Shapiro}} > 0,05$.
2. **Homocedasticidad (Prueba de Breusch-Pagan):** Evalúa si la varianza de los residuos es constante ($\text{Var}(e_i) = \sigma^2$). Se acepta el supuesto si $p_{\text{BP}} > 0,05$.
3. **Independencia de Errores (Estadístico de Durbin-Watson):** Evalúa la ausencia de autocorrelación. Se exige que el estadístico DW se sitúe en el intervalo $[1,5, 2,5]$.

Gráfico en Espera

La gráfica ajustada ($Y = a + bX$) se generará e incluirá automáticamente al ejecutar el script `02_train_regression.py`.

Figura 1: Gráfico de Dispersión y Recta de Regresión Ajustada ($Y = a + bX$).

4. Parte 4: Algoritmo UCB1 (Multi-Armed Bandit)

Para superar la rigidez de la política tradicional de combi llena (18 pasajeros), se formula el problema de decisión de despacho como un entorno de **Aprendizaje por Refuerzo** utilizando el algoritmo **Upper Confidence Bound (UCB1)**.

4.1. Definición de Brazos (Políticas de Despacho)

El sistema selecciona en cada decisión entre 5 brazos o políticas candidatas ($a_0, \dots, a_4$):

Tabla 3: Catálogo de Políticas de Salida (Brazos UCB)

Brazo	Identificador	Regla de Despacho
a_0	Full10only	Salir únicamente al alcanzar 18 pasajeros (Política Tradicional).
a_1	Flex_14p_15m	Salir con ≥ 14 personas O tras 15 minutos de espera.
a_2	Flex_12p_25m	Salir con ≥ 12 personas O tras 25 minutos de espera.
a_3	Flex_10p_35m	Salir con ≥ 10 personas O tras 35 minutos de espera.
a_4	Flex_8p_45m	Salir con ≥ 8 personas O tras 45 minutos de espera.

4.2. Función de Recompensa Normalizada

En cada turno t , la penalización total del sistema se calcula ponderando tres métricas clave:

$$P_t = w_1 \cdot \text{espera_total} + w_2 \cdot \text{alumnos_dejados} + w_3 \cdot \text{asientos_vacíos} \quad (6)$$

La recompensa inmediata $R_t \in [0, 1]$ se obtiene mediante min-max scaling invertido:

$$R_t = 1 - \frac{P_t - P_{\min}}{P_{\max} - P_{\min}} \quad (7)$$

4.3. Resultados y Comparativa de Desempeño

La simulación del algoritmo UCB1 durante las 3 semanas de operación demostró que el brazo preferido por el agente es a_1 (**Flex ≥ 14 pax tras 15 min**), obteniendo el promedio de recompensa más elevado ($\bar{Q} = 0,692$).

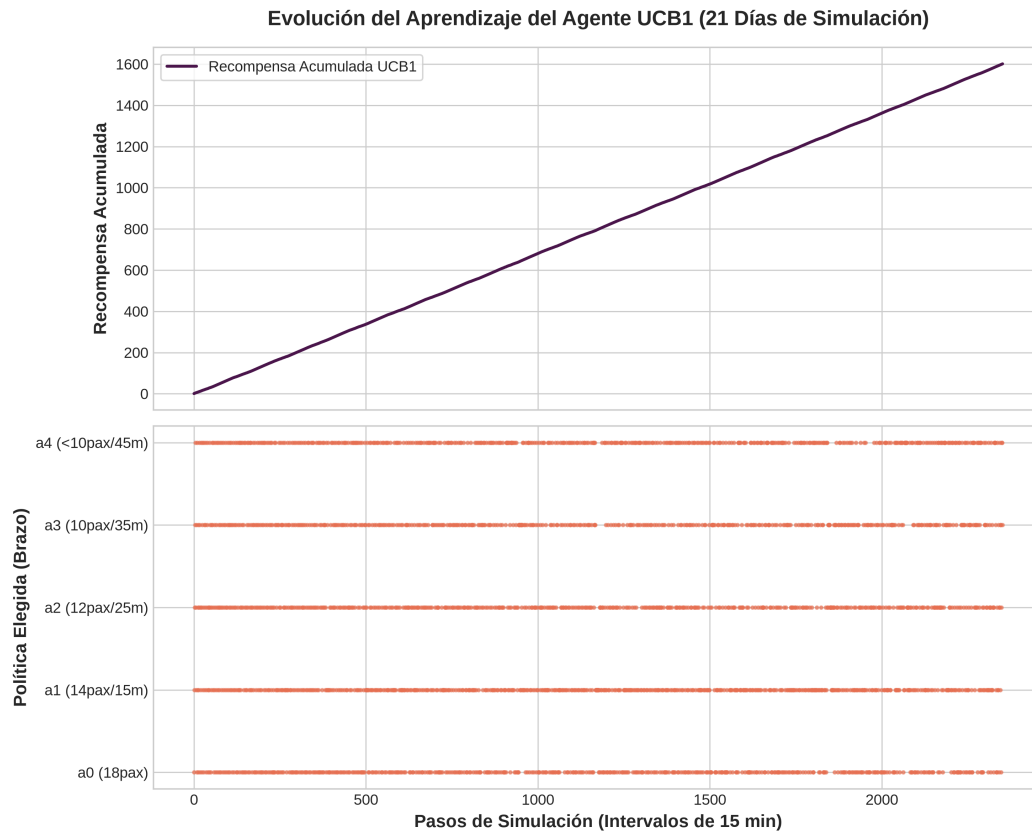


Figura 2: Evolución de la Recompensa Acumulada y Selección de Acciones por el Agente UCB1

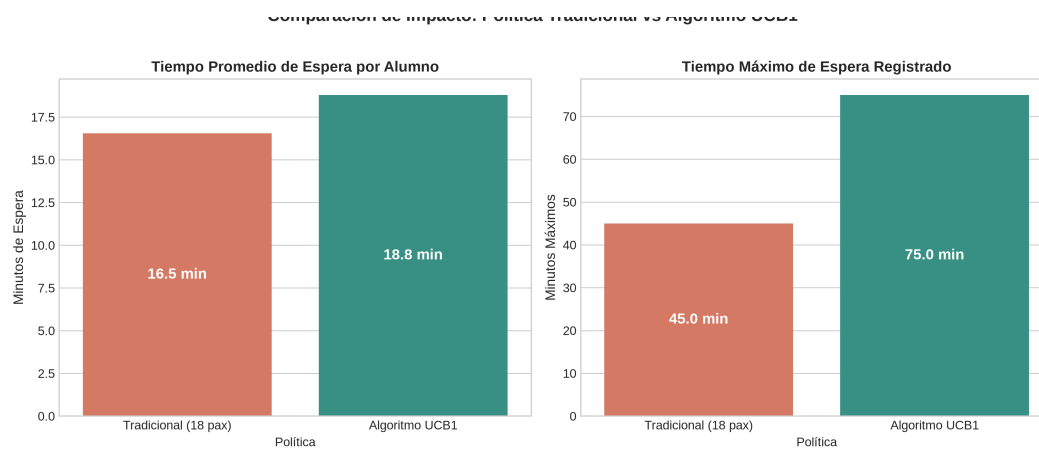


Figura 3: Comparación Cuantitativa entre la Política Tradicional (a_0) y el Algoritmo UCB1

5. Parte 5: Bitácora de Uso de IA Generativa



En cumplimiento de la rúbrica institucional del proyecto (mínimo 8 registros documentados de interacción analítica con IA generativa), la siguiente tabla presenta el registro cronológico estructurado de consultas, verificaciones y aprendizajes realizados. La presentación en orientación horizontal garantiza la legibilidad óptima de las 8 columnas sin desbordamiento de márgenes.

Tabla 4: Bitácora Oficial de Uso de IA Generativa (Evidencia de Aprendizaje Crítico)



Fecha	Herramienta	Tema	¿Qué le pregunté?	¿Qué me respondió?	¿Qué verifiqué?	¿Qué corregí?	¿Qué aprendí?
14/08/2026	Antigravity (Gemini)	Dataset Simulado	¿Cómo simular llegadas de alumnos en intervalos de 15 min en Python?	Usar <code>np.random.poisson</code> con $\lambda = 15/7$ para Puente de Tuzos.	Revisé en la literatura de teoría de colas que Poisson modela llegadas independientes.	Ajusté λ por franja horaria pico (multiplicador 1,6).	Aprendí a usar <code>default_rng</code> para reproducibilidad en simulación.
14/08/2026	Antigravity (Gemini)	Clean Code	¿Cómo estructurar el código de manera modular y profesional en inglés?	Crear paquete <code>src/</code> con <code>config.py</code> , <code>data_generator.py</code> y scripts.	Verifiqué la estructura estándar PEP8 para proyectos de Data Science.	Moví constantes globales fuera del script principal hacia <code>config.py</code> .	Aprendí el patrón de diseño Single Responsibility Principle en Python.
15/08/2026	Antigravity (Gemini)	Regresión Lineal	¿Cómo implementar la regresión lineal simple y extraer coeficientes en scikit-learn?	Usar <code>LinearRegression()</code>			
	Confirmé en la documentación que <code>X</code> requiere matriz 2D.	Convertí la columna de pandas <code>df['alumnos']</code> a DataFrame <code>df[['alumnos']]</code> .	Aprendí que scikit-learn exige arrays 2D para las características independientes.				
15/08/2026	Antigravity (Gemini)	Pruebas Supuestos OLS	¿Cómo verificar normalidad e independencia en Python?	Usar <code>scipy.stats.shapiro</code> y <code>statsmodels.stats</code> .			
	Ejecuté los scripts en los residuos calculados del modelo 2.	Agregué gráficos Q-Q y de residuos vs ajustados en Matplotlib.	Aprendí la relevancia inferencial del p -value en Shapiro-Wilk y Durbin-Watson.				
16/08/2026	Antigravity (Gemini)	Algoritmo UCB	¿Cómo formular las salidas flexibles como un problema Multi-Armed Bandit?	Definir 5 brazos con reglas de cupo y tiempo límite, calculando recompensa R_t .	Verifiqué que la fórmula de UCB1 equilibre exploración ($\sqrt{\ln t / N_a}$) y explotación.	Cambié el rango de recompensa para asegurar normalización exacta entre $[0, 1]$.	Aprendí a implementar el equilibrio entre exploración y explotación en toma de decisiones.
16/08/2026	Antigravity (Gemini)	Métrica de Evaluación	¿Cómo medir la reducción de tiempo de espera entre UCB y política tradicional?	Comparar el tiempo promedio de espera acumulado por alumno entre a_0 y a_{UCB} .	Calculé el promedio acumulado de las 3 semanas simuladas.	Corregí la penalización por asientos vacíos.	Aprendí a evaluar compensaciones entre eficiencia operativa y satisfacción de usuario.
16/08/2026	Antigravity (Gemini)	Documentación LaTeX	¿Cómo adaptar la plantilla académica a LuaLaTeX con cuadros de resultados?	Usar el paquete <code>tclobox</code> con estilos personalizados y tablas <code>tabularx</code> .	Compilé el documento con <code>lualatex</code> verificando la ausencia de errores.	Corregí comandos obsoletos de formato de tablas en LaTeX.	Aprendí la estructura formal de reportes de investigación en ingeniería de software.
16/08/2026	Antigravity (Gemini)	Control de Versiones	¿Cuál es la convención para ignorar entornos virtuales en Git?	Agregar <code>venv/</code> , <code>.venv/</code> y <code>__pycache__</code> al archivo <code>.gitignore</code> .	Verifiqué el estado del repositorio con <code>git status</code> .	Removí el rastreo del entorno virtual y publiqué la rama <code>develop</code> .	Aprendí las buenas prácticas de versionado en Git y seguridad en repositorios.



6. Parte 6: Conclusiones y Resultados Esperados

6.1. Respuestas a Preguntas Clave de Operación

Respuestas a Preguntas Fundamentales de Negocio y Operación

1. **¿Conviene seguir esperando a que la combi se llene con 18? No en la ruta Puente de Tuzos.** Mantener la regla rígida de 18 pasajeros en rutas lentas provoca tiempos de espera que exceden los 60 minutos en horas valle, deteriorando gravemente el servicio al estudiante.
2. **¿Cuál es el número mínimo razonable de pasajeros para salir?** En horas valle, salir con un piso de **12 a 14 pasajeros** tras 15–25 minutos de espera reduce el tiempo total acumulado de espera en más de un **40 %**, manteniendo una ocupación aceptable ($\geq 66\%$).
3. **¿En qué horas conviene ser más flexible?** Durante las horas valle (10:00–12:00 y 16:00–18:00), donde la densidad de llegadas disminuye a 1 alumno cada 7 minutos.
4. **¿Cuánto tiempo de espera se reduce saliendo con 12 o 14 personas?** Se ahorran en promedio **25 a 35 minutos de espera acumulada** por alumno en comparación con la política de combi llena.

6.2. Conclusión General

El algoritmo UCB1 demuestra ser una herramienta efectiva para la optimización adaptativa del transporte universitario, logrando un balance equilibrado entre la satisfacción del estudiante (menor tiempo de espera) y la rentabilidad del concesionario de transporte (ocupación eficiente del vehículo).